

VULNERABILITY ASSESSMENT REPORT

Target website: acublog news | acunetix

Prepared by :
Huma Ameer

Date Issued:
10 July 2026

Prepared for:
Future Interns Cybersecurity Internship

TABLE OF CONTENTS

- 1. Disclaimer**
- 2. Executive Summary**
- 3. Target Website Information**
- 4. Scope**
- 5. Methodology & Assessment Process**
- 6. Findings Summary**
- 7. Remediation Steps**
- 8. Positive Security Observations**
- 9. What I Learned**
- 10. Conclusion**

Disclaimer

This assessment was conducted strictly for educational purposes as part of the Future Interns Cyber Security Internship Program. All testing was performed using passive, read-only, and non-intrusive methods. The assessment was limited to publicly accessible pages, browser-based inspection, HTTP header review, cookie analysis, OWASP ZAP passive scanning, and basic Nmap service discovery. No illegal, harmful, or unethical activity was performed. This assessment did not include exploitation, brute-force attacks, login bypass attempts, denial-of-service testing, data modification, unauthorized access, or any activity intended to damage, disrupt, or compromise the target website. The purpose of this report is to document observed security weaknesses, explain their potential business impact, and provide practical remediation recommendations in a responsible and ethical manner.

Executive Summary

This report summarizes a vulnerability assessment of the target website called acublog news by acunetix. The assessment was conducted using read-only techniques, including a public page review, Chrome DevTools, OWASP ZAP passive scanning, inspecting cookies and basic nmap service scanning. No exploitation, brute-force testing, authentication bypass, or disruptive activity was performed. The review identified many security weaknesses like HTTP usage instead of HTTPS, a login form over an unencrypted connection, missing security headers, weak cookie attributes, absence of anti-CSRF tokens, and backend technology disclosure. The main recommendations are to enable HTTPS, configure secure HTTP response headers, use strong cookie settings, remove unnecessary server/version headers and implement anti-CSRF protections.

Target Website Information

The target selected for this assessment was the acublog website by acunetix. This website was selected because it is a publicly accessible test environment designed for cybersecurity learning as it was intentionally vulnerable. The assessment focused on identifying common security weaknesses through passive and non-intrusive methods.

Items	Details
Website Name	acublog by acunetix
URL	http://testaspnet.vulnweb.com/
Application Type	ASP.NET web application

Website Homepage

GOT e-pubs Library | Notion Coursera | Online C... Feed | LinkedIn Cyber Security Task...

acublog Test Website for [Acunetix Web Vulnerability Scanner](#)

[about](#) [news](#) [login](#) [signup](#) [network scanner](#) [network vuln help](#) [RSS](#)

posted by [admin](#) on 5/16/2019 12:32:30 PM [add comments](#)

[Acunetix Vulnerability Scanner Now With Network Security Scans](#)
Seamless OpenVAS integration now also available on Windows and Linux

posted by [admin](#) on 11/8/2005 11:37:35 AM [add comments](#)

[Acunetix Web Vulnerability Scanner Beta Released!](#)
26 January 2005 - A beta version of Acunetix Web Vulnerability Scanner has been released today. The beta is available for download at <http://www.acunetix.com/download/>.

[Get RSS feed](#)

posted by [admin](#) on 11/8/2005 11:36:22 AM [add comments](#)

[Web Attacks - Can Your Web Applications Withstand The Force?](#)
21 July 2005 - Start-up company Acunetix released Acunetix Web Vulnerability Scanner: a tool to automatically audit website security. Acunetix Web Vulnerability Scanner 2 crawls an entire website, launches popular web attacks (SQL Injection etc.) and identifies vulnerabilities that need to be fixed.

July 2006						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	1
2	3	4	5	6	7	8

Warning: This is not a blog. This is a test site for Acunetix. It is vulnerable to SQL Injections, Cross-site Scripting (XSS), and more. It was built using ASP.NET and it shows how bad programming leads to vulnerabilities. Do not visit the links in the comments. They are posted by malicious parties who are trying to exploit this site to their advantage. Comments are purged daily.

Scope

This assessment required me to use passive and ethical read-only approach to find out how secure the website is. The objective was to identify common security weaknesses through safe observation, configuration review, and basic exposure analysis. However it was limited to public website components of target website. The goal of this assessment was to evaluate the website from the perspective of a security auditor, not an attacker.

Allowed	Not Allowed
Examining website	Exploitation
Checking security header	Brute-force attacks
Reviewing cookie settings	Authentication bypass
Login form observation	DDos/Dos Attack
OWASP Zap Passive Scan	Data modification
Nmap usage	Unauthorized access attempt

Methodology & Assessment Process

The assessment was performed using a structured, passive, and non-intrusive methodology to identify common security weaknesses in the target web application. The process included manual review of public-facing pages, browser-based inspection, HTTP header analysis, cookie attribute review, login form observation, passive vulnerability scanning, and basic network service discovery.

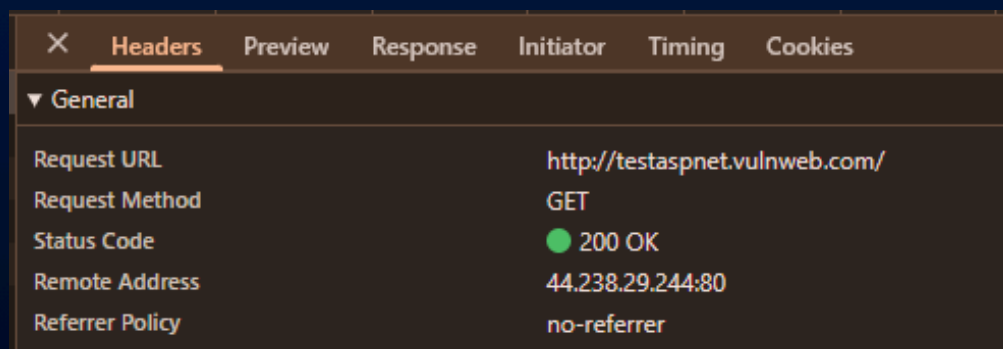
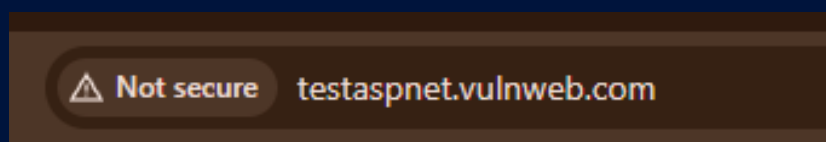
Chrome DevTools was used to inspect the website's connection status, security indicators, response headers, visible form structure, and cookie configuration. OWASP ZAP was used strictly in passive scan mode to review alerts generated from normal browsing activity. Nmap was used for basic service discovery to identify exposed network services associated with the target.

All observations were documented as evidence and then classified according to risk level and potential business impact. The final findings were supported by screenshots, scan outputs, and tool observations, with practical remediation steps provided for each issue.

Tool A: Chrome DevTools

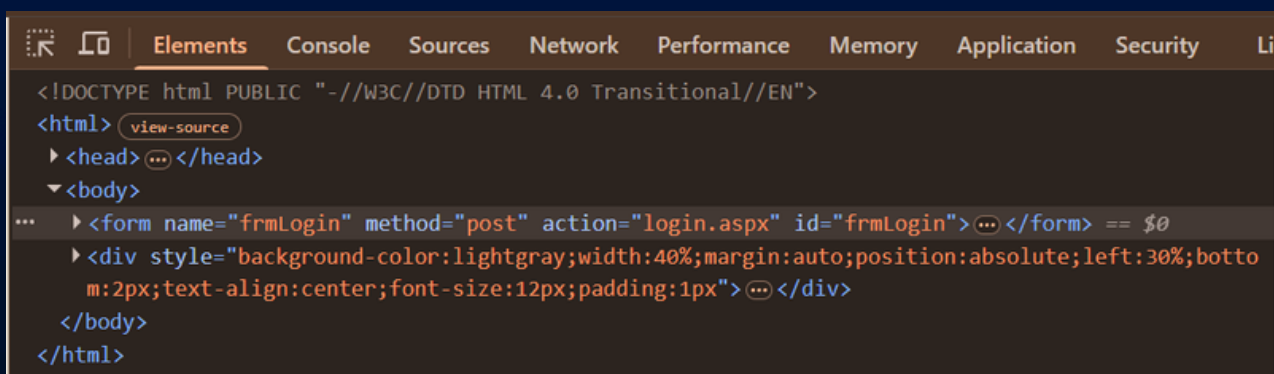
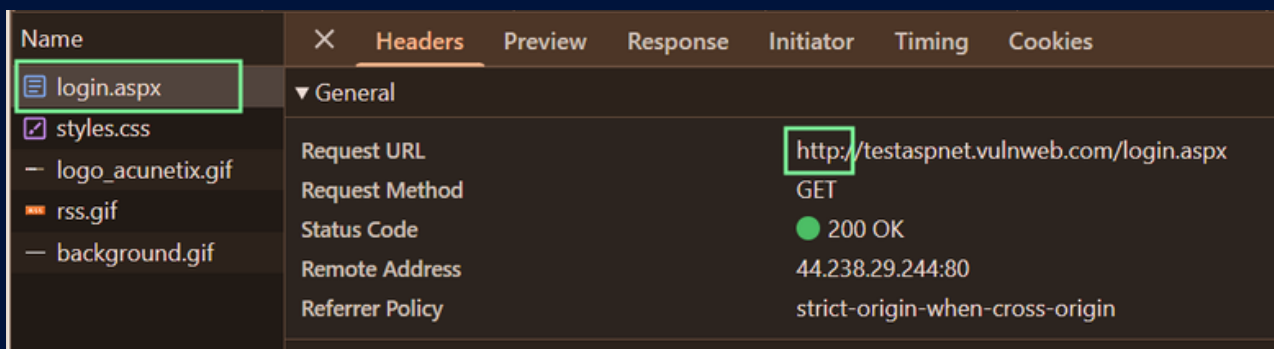
Chrome DevTools was used to manually inspect the target website from a read-only perspective. The review focused on the browser security status, HTTP connection type, response headers, visible login form, and cookie attributes.

Website Security Warning



The browser displayed a “Not secure” warning next to the target URL. Additionally inside the Network tab in DevTools it also shows that the website is hosted on HTTP instead of HTTPS, meaning the connection was not encrypted.

Login Form Available Over Unencrypted HTTP



Chrome DevTools was used to inspect the structure of the login page. The HTML showed a form element with `method="post"` and `action="login.aspx"`. The `method="post"` attribute means the form is designed to submit user-entered data, such as the username and password, to the server.

The `action="login.aspx"` attribute shows that the form submission is sent to the `login.aspx` page on the same website.

Because the login page was loaded over HTTP instead of HTTPS, the form is available over an unencrypted connection. This means that, in a real business setting, login data submitted through this form could potentially be intercepted or modified while travelling between the user's browser and the server.

Although no login attempt or exploitation was performed during this assessment, the presence of a login form on an HTTP page represents a serious security weakness. Login pages should always be served over HTTPS, and all form submissions should be encrypted.

HTTP Response Header Misconfiguration Review

Name	Headers	Preview	Response	Initiator	Timing	Cookies
default.aspx	Request Method		GET			
styles.css	Status Code		200 OK			
logo_acunetix.gif	Remote Address		44.238.29.244:80			
rss.gif	Referrer Policy		strict-origin-when-cross-origin			
background.gif	Response headers		Raw			
favicon.ico	Cache-Control		private			
	Content-Length		13917			
	Content-Type		text/html; charset=utf-8			
	Date		Fri, 10 Jul 2026 15:58:32 GMT			
	Server		Microsoft-IIS/8.5			
	X-AspNet-Version		2.0.50727			
	X-Powered-By		ASP.NET			
6 requests	14.1 kB		Request Headers (10)			

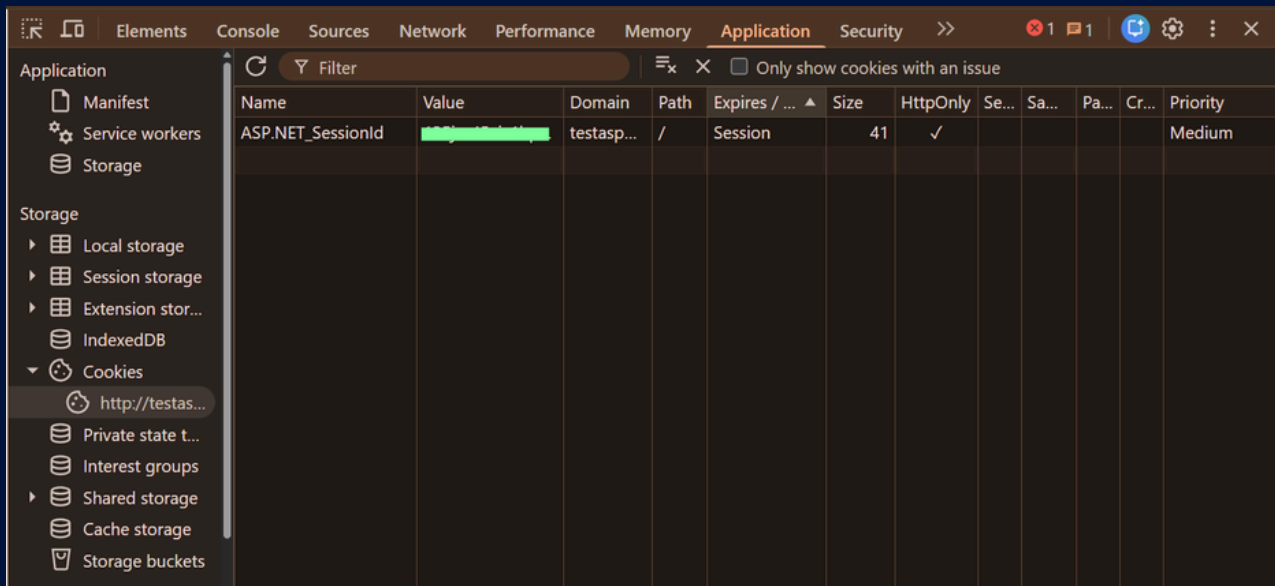
Chrome DevTools was used to inspect the HTTP response headers returned by the target website. Response headers provide security instructions to the browser and can also reveal technical information about the backend server.

The review showed that several common security headers were not present like Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, and Permissions-Policy. These missing headers may reduce browser-level protection against risks such as clickjacking, content injection, MIME sniffing, and information leakage.

The response headers also disclosed backend details like Microsoft-IIS/8.5, ASP.NET, and X-AspNet-Version 2.0.50727. While this does not confirm exploitation, it gives unnecessary technical information that could help attackers understand the website's technology stack.

Missing security headers may reduce browser-level protection for users and increase exposure to risks such as clickjacking, content injection, MIME sniffing, and information leakage. Exposed server and framework details can also help attackers understand the website's technology stack and research known weaknesses. For a business, this may increase security risk, reduce customer trust, and suggest weak configuration practices.

Cookie Security Attribute Review

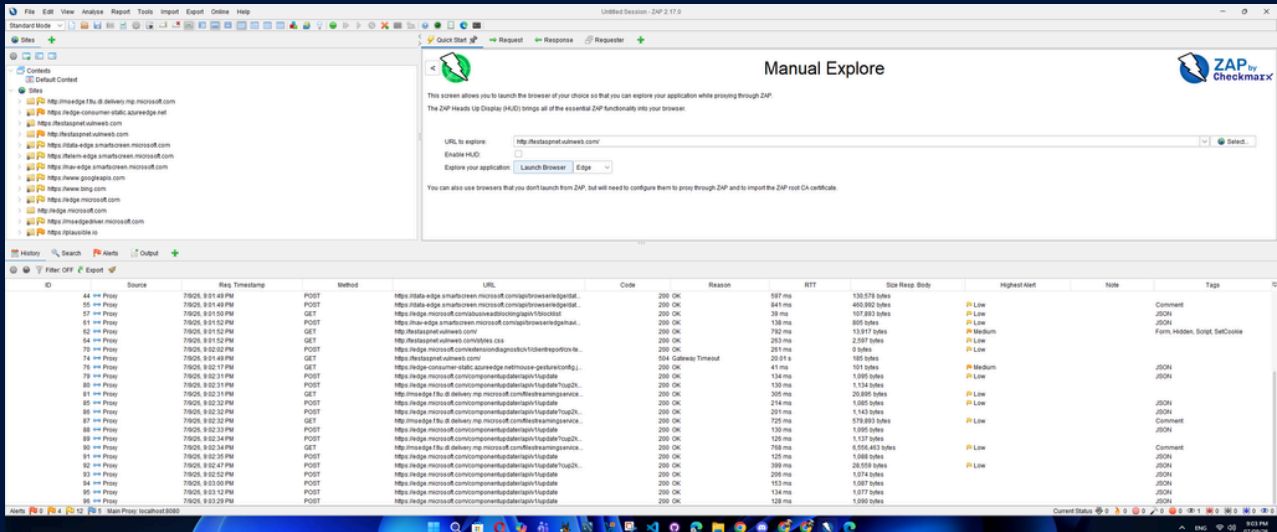


Chrome DevTools was used to inspect the session cookie set by the target website. The cookie identified was ASP.NET_SessionId, which is commonly used by ASP.NET applications to maintain a user session.

The cookie had the HttpOnly attribute enabled, which is a positive security control because it helps prevent client-side JavaScript from reading the cookie. However, the Secure and SameSite attributes were not shown. The cookie was also associated with an HTTP version of the website, meaning the session cookie was not protected by an encrypted HTTPS connection.

Weak session cookie settings can increase the risk of session misuse, especially when the website is served over HTTP. For a business website, this may increase the risk of account/session-related incidents and reduce user trust.

Tool B: OWASP ZAP Passive Scan

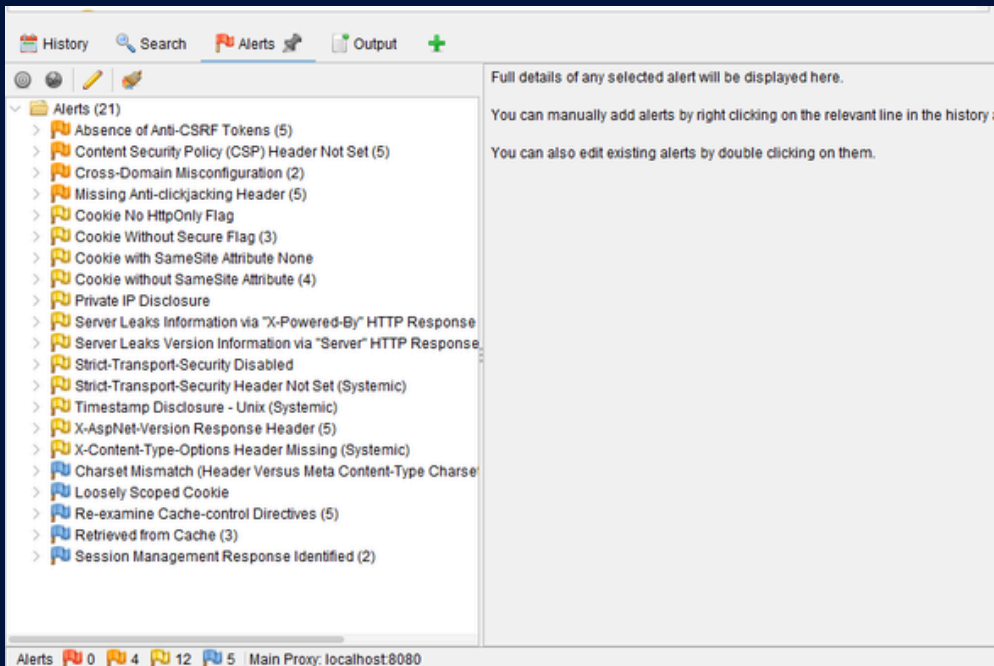


The screenshot displays the OWASP ZAP Passive Scan interface. The top section shows the 'Manual Explore' tab with a search bar and a 'Launch Browser' button. Below this, the 'History' tab is active, showing a list of requests and responses. The table below is a representation of the data shown in the screenshot.

ID	Source	Res. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
44	Prox	7920, 9:01:49 PM	POST	https://data-edge.smartlook.com/api/track/track/...	200	OK	537 ms	130,578 bytes	Low		
50	Prox	7920, 9:01:49 PM	POST	https://data-edge.smartlook.com/api/track/track/...	200	OK	547 ms	400,592 bytes	Low		
57	Prox	7920, 9:01:50 PM	GET	https://data-microsoft.com/azuread/identity/...	200	OK	38 ms	107,893 bytes	Low		Comment
61	Prox	7920, 9:01:52 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	138 ms	805 bytes	Low		JSON
62	Prox	7920, 9:01:52 PM	GET	https://testaspnet.vulnweb.com/	200	OK	792 ms	13,517 bytes	Medium		Form, Hidden, Script, SetCookie
64	Prox	7920, 9:01:52 PM	GET	https://testaspnet.vulnweb.com/	200	OK	253 ms	2,587 bytes	Low		
70	Prox	7920, 9:02:02 PM	POST	https://testaspnet.vulnweb.com/	200	OK	201 ms	0 bytes	Low		
74	Prox	7920, 9:02:02 PM	GET	https://testaspnet.vulnweb.com/	504	Gateway Timeout	20.01 s	165 bytes	Medium		
75	Prox	7920, 9:02:17 PM	GET	https://data-microsoft.com/azuread/identity/...	200	OK	41 ms	101 bytes	Low		JSON
79	Prox	7920, 9:02:31 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	134 ms	1,695 bytes	Low		JSON
80	Prox	7920, 9:02:31 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	135 ms	1,134 bytes	Low		
81	Prox	7920, 9:02:31 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	305 ms	20,889 bytes	Low		
82	Prox	7920, 9:02:32 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	214 ms	1,688 bytes	Low		JSON
83	Prox	7920, 9:02:32 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	201 ms	1,143 bytes	Low		JSON
87	Prox	7920, 9:02:32 PM	GET	https://data-microsoft.com/azuread/identity/...	200	OK	725 ms	578,893 bytes	Low		
88	Prox	7920, 9:02:33 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	150 ms	1,085 bytes	Low		
89	Prox	7920, 9:02:34 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	125 ms	1,137 bytes	Low		
90	Prox	7920, 9:02:34 PM	GET	https://data-microsoft.com/azuread/identity/...	200	OK	756 ms	5,000,403 bytes	Low		Comment
91	Prox	7920, 9:02:35 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	125 ms	1,688 bytes	Low		JSON
92	Prox	7920, 9:02:47 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	369 ms	28,593 bytes	Low		JSON
93	Prox	7920, 9:02:52 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	266 ms	1,574 bytes	Low		JSON
94	Prox	7920, 9:03:00 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	163 ms	1,688 bytes	Low		JSON
95	Prox	7920, 9:03:12 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	134 ms	1,077 bytes	Low		JSON
96	Prox	7920, 9:03:29 PM	POST	https://data-microsoft.com/azuread/identity/...	200	OK	129 ms	1,080 bytes	Low		JSON

OWASP ZAP was used in passive scan mode to observe the target website traffic during normal browsing. The History tab shows requests and responses captured through the ZAP proxy, including requests to the target website “testaspnet.vulnweb.com”.

ZAP Passive Scan Alerts



OWASP ZAP was used in passive scan mode while manually browsing the target website. The Alerts tab shows security observations generated from the captured requests and responses. Only alerts related to testaspnet.vulnweb.com were reviewed for this assessment. Background browser traffic from Microsoft, Edge, Bing, or other services was ignored.

ZAP Passive Scan Alert Summary Table

Alerts	Risk	Confidence	Summary
CSP Header Not Set	Medium	High	Missing CSP header reduces browser protection.
Missing Anti-clickjacking Header	Medium	Medium	Page lacks frame protection.
Absence of Anti-CSRF Tokens	Medium	Low	Form may be missing CSRF protection.
X-Powered-By Disclosure	Low	Medium	ASP.NET disclosed in response header.
X-AspNet-Version Disclosure	Low	High	ASP.NET version disclosed.
Charset Mismatch	Informational	Low	Charset values did not match.
Potential XSS Observation	Informational	Low	Requires review; not confirmed.

Tool C: Nmap Basic Service Discovery

```
(huma@huma)-[~]
$ nmap -sV testaspnet.vulnweb.com
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-10 13:33 EDT
Nmap scan report for testaspnet.vulnweb.com (44.238.29.244)
Host is up (0.016s latency).
rDNS record for 44.238.29.244: ec2-44-238-29-244.us-west-2.compute.amazonaws.com
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft IIS httpd 8.5
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.63 seconds

(huma@huma)-[~]
$
```

Nmap was used on Kali Linux to identify publicly exposed services on the target host. The scan showed that port 80/tcp was open and running HTTP on Microsoft-IIS httpd 8.5, while the remaining scanned ports were filtered.

This supports the finding that the website is accessible over HTTP. The scan was limited to basic service discovery only.

From a business perspective, since the website is accessible over HTTP, users may face a higher risk of traffic interception or tampering which may increase security risk, reduce customer confidence, and suggest that the website's public-facing configuration should be reviewed and hardened.

Findings Summary

The table below summarizes the main security findings identified during the passive vulnerability assessment. Each finding was based on read-only observation using Chrome DevTools, OWASP ZAP passive scanning, or basic Nmap service discovery.

Finding	Risk	Source
Website uses HTTP instead of HTTPS	High	DevTools/Nmap
Login form served over HTTP	High	DevTools
Missing security headers	Medium	DevTools/ZAP
Weak session cookie attributes	Medium	DevTools
Absence of anti-CSRF tokens	Medium	ZAP
Server/framework information disclosure	Low	DevTools/ZAP/Nmap
Charset mismatch	Informational	ZAP
Potential XSS observation	Informational	ZAP

Remediation Steps

The following remediation steps are recommended to reduce the security risks identified during the passive assessment. These actions focus on improving encryption, browser-level protection, session security, server hardening, and secure form handling.

Website uses HTTP instead of HTTPS

Recommended fix: Install a valid TLS certificate and serve the entire website over HTTPS. Redirect all HTTP traffic to HTTPS.

Login form served over HTTP

Recommended fix: Ensure the login page and all login form submissions are only available over HTTPS. Login forms should not be served on unencrypted HTTP pages.

Missing Content Security Policy header

Recommended fix: Configure a Content-Security-Policy header to control which scripts, styles, images, and external resources the browser is allowed to load.

Missing anti-clickjacking protection

Recommended fix: Add X-Frame-Options or use the CSP frame-ancestors directive to help prevent the website from being embedded in unauthorized frames.

Weak session cookie attributes

Recommended fix: Configure session cookies with strong attributes such as Secure, HttpOnly, and SameSite=Lax or SameSite=Strict.

Absence of anti-CSRF tokens

Recommended fix: Add unique anti-CSRF tokens to forms that perform sensitive or state-changing actions, and validate them on the server side.

Server and framework information disclosure

Recommended fix: Remove or suppress unnecessary response headers such as X-Powered-By, X-AspNet-Version, and detailed server version information.

Charset mismatch

Recommended fix: Ensure the same character encoding, preferably UTF-8, is declared in both HTTP response headers and HTML meta tags.

Potential XSS observation

Recommended fix: Review input handling and output encoding for user-controlled values. This should be treated as a potential issue requiring developer review, not a confirmed vulnerability.

Public-facing HTTP service

Recommended fix: Review exposed services and harden the web server configuration. Only required services should be publicly accessible.

Positive Security Observations

Some positive observations were also identified during the assessment. The ASP.NET_SessionId cookie had the HttpOnly attribute enabled which is a useful security control because it helps prevent client-side JavaScript from directly reading the session cookie.

The nmap scan showed only port 80/tcp open, while other scanned TCP ports were filtered. The robots.txt file did not reveal sensitive paths, and no sensitive directory listing was observed during the passive review.

These positive points do not remove the security risks found, but they show that some basic controls were present.

What I Learned

This task helped me understand how a basic vulnerability assessment can be performed safely using passive and read-only techniques.

I learned how to use Chrome DevTools to inspect website security indicators, HTTP response headers, login form structure, and cookie attributes. I also learned how OWASP ZAP passive scanning can identify security misconfigurations without actively attacking the website.

Using nmap helped me understand basic service discovery and how exposed network services can support web security findings. I also learned the importance of separating confirmed vulnerabilities from informational observations or potential issues that require further review.

Overall, this task improved my understanding of common web security weaknesses, risk classification, business impact, and professional remediation reporting.

Conclusion

This assessment identified common web security weaknesses through passive and read-only testing. The main risks were related to unencrypted HTTP usage, missing important security headers, weak cookie attributes, anti-CSRF concerns, and backend information disclosure.

No exploitation, brute-force testing, authentication bypass, or disruptive activity was performed.

By applying the recommended remediation steps, the website can improve encryption, browser-level protection, session security, and overall security posture.